Co-Occurrence Matrices

Machine Learning on Text

- Spam Filter:

Greetings,

I seek for a trusted, sincere and honest person to judiciously set up a charity foundation under my sponsorship.

The foundation will be strictly for helping the poorest of the poor; especially the orphanage homes and homes for destitute and the mentally retarded.

I am 76 years old and suffering from Protracted Cancer Of The Lungs and on admission at the specialist hospital intensive care unit.

My lawyer that was supposed to handle the process of executing my Will & Testament was shot during the protest against the presidential election.

If t touched your spirit to be part in putting smile and new life on the vulnerable, kindly reply through my personal below email for more information.

Mrs. Matsushita

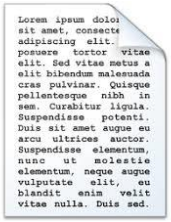
Email: matsumorand@gmail.com

Formulaire sans titre

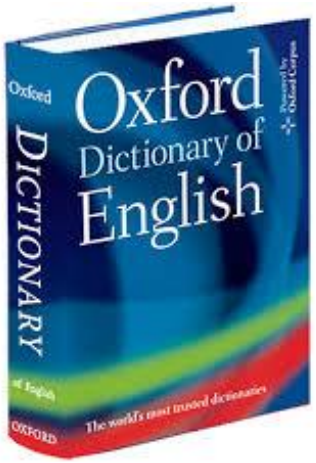
REMPLIR LE FORMULAIRE

Bag of Words

- Goal: Represent text as a vector.

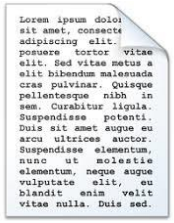


"Viagra Oral Jelly is a remedy that helps to control and restore potency during sexual arousal. The main advantage of Viagra Oral Jelly is its fast action"



- Bag of Words:
  - Assume all words in text come from the English dictionary.

Vector has 1 entry per word in dictionary





"Viagra"

(0,1,0,0,…,2,0,1,0)

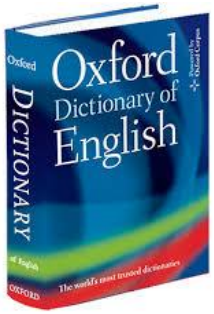
"potency" "cow"

Entry stores count of many times word occurs in text.

TF-IDF



- Bag-of-words:
  - Entry t in vector for document d is:
    - $f_{t,d}$ = #occurrences of t in d
- Tf-idf:
  - Entry t in vector for document d is:
    - $tf(t,d) \cdot idf(t) = \frac{f_{t,d}}{|d|} \cdot \ln\left(\frac{n}{|\{d_i : t \in d_i\}|}\right)$



- Issue:
  - Similarity between words ignored:
    - ”man”, ”boy”,”woman”,”girl” not treated differently from ”apple”,”football” and ”this”.

Word Embeddings



- Bag-of-words:
  - Entry t in vector for document d is:
    - $f_{t,d}$ = #occurrences of t in d

- Formalizing the issue:
  - Think of each word as being represented by a vector.
  - In Bag-of-words:
    - Each word t corresponds to a standard unit vector $w(t)=e_t$.
    - Document d represented as vector v(d):
      - $v(d) = \sum_{t \in d} w(t)$

Word Embeddings



- Bag-of-words:
  - Each word t corresponds to a standard unit vector $w(t)=e_t$.
  - Document d represented as vector v(d):
    - $v(d) = \sum_{t \in d} w(t)$
- *"partner abroad who can accommodate 200M USD for Investment"*

"for" "can" "who" "abroad" "partner"

v(d) (0,1,0,0,…,…..,0,1,0,0,….,1,0,……0,1,0,…….,0,0,1,…….)

$e_{\text{"for"}}$ (0,1,0,0,…,…..,0,0,0,0,….,0,0,……0,0,0,…….,0,0,0,…….)

$e_{\text{"can"}}$ (0,0,0,0,…,…..,0,1,0,0,….,0,0,……0,0,0,…….,0,0,0,…….)

$e_{\text{"who"}}$ (0,0,0,0,…,…..,0,0,0,0,….,1,0,……0,0,0,…….,0,0,0,…….)

Word Embeddings

- Bag-of-words:
  - Each word t corresponds to a standard unit vector w(t)=e_t.
  - Document d represented as vector v(d):
    - $v(d) = \sum_{t \in d} w(t)$
- Issue:
  - Similarity between words ignored:
    - ”man”, ”boy”,”woman”,”girl” not treated differently from ”apple”,”football” and ”this”.
    - $w(\text{"man"})^T w(\text{"boy"}) = 0$
    - $w(\text{"man"})^T w(\text{"this"}) = 0$
    - $\|w(\text{"woman"}) - w(\text{"girl"})\| = \|w(\text{"boy"}) - w(\text{"apple"})\|$

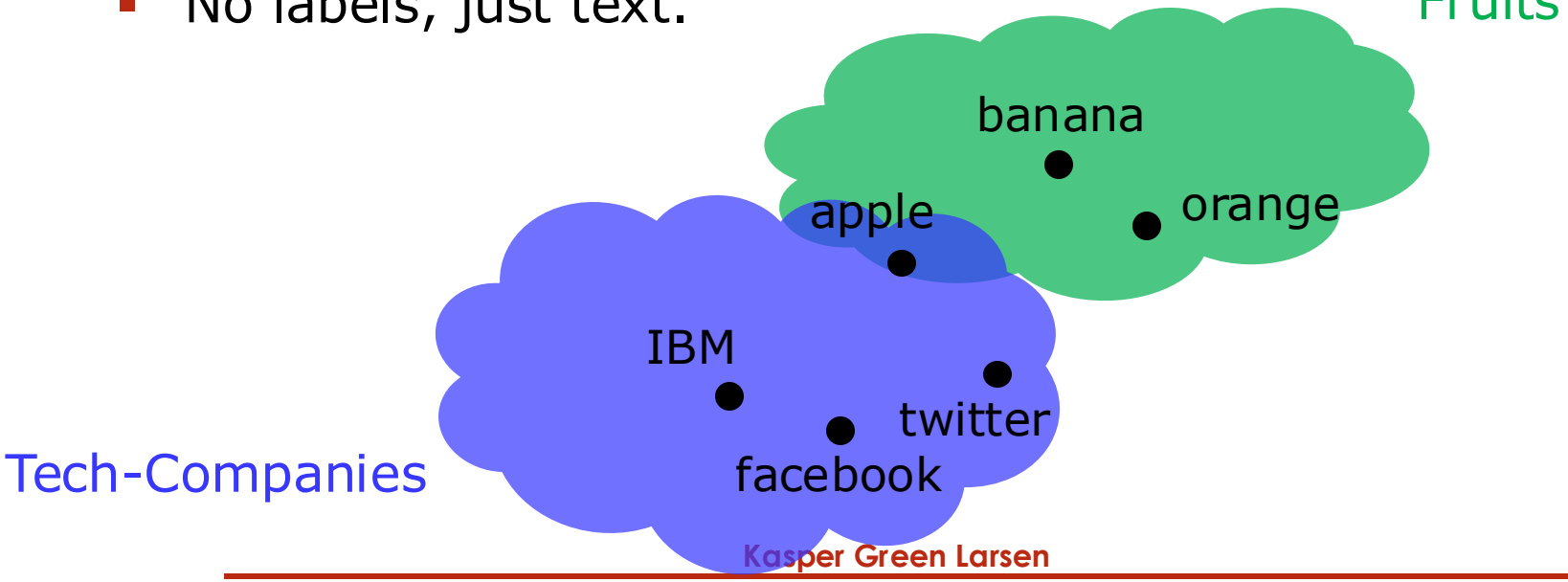
Word Embeddings

- Bag-of-words:
  - Each word t corresponds to a standard unit vector $w(t)=e_t$.
  - Document d represented as vector v(d):
    - $v(d) = \sum_{t \in d} w(t)$
- Issue:
  - Similarity between words ignored.
  - Would be sensible if v(d) was similar to v(d') if replace all words in d with synonyms to obtain d'.
- Question:
  - How can we define word vectors that "preserve" "similarity" between words?

Word Embeddings

- Question:
  - How can we define word vectors that ”preserve” ”similarity” between words?

- Unsupervised Learning!
  - Learn similarities between words from data.
  - No labels, just text.



Word Embeddings

- Co-Occurrence Matrix:
  - Base similarity on occurring in similar sentence structures / together with similar words.

Co-occurrences:
("who","partner")
("who","abroad")
("who","can")
("who","accommodate")

- Two concepts: Co-occurrence and context window.

context window

- *"partner abroad who can accommodate 200M USD for Investment"*

- Context window: Specified by a width around a word.
  - Example: Context window of width 2 around "who".
- Co-Occurrence: Have a co-occurrence between "who" and every word in the context window around "who".

Word Embeddings

- Co-Occurrence Matrix:
  - Base similarity on occurring in similar sentence structures / together with similar words.

Co-occurrences:
("can","abroad")
("can","who")
("can","accommodate")
("can","200M")

- Two concepts: Co-occurrence and context window.

context window

- *"partner abroad who can accommodate 200M USD for Investment"*

- Context window: Specified by a width around a word.
  - Example: Context window of width 2 around "can".
- Co-Occurrence: Have a co-occurrence between "can" and every word in the context window around "can".

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	…	1	0
taste	1	0	…	2	1
...	…	…	…	…	…
good	1	2	…	0	0
pears	0	1	…	0	0

Co-occurrence matrix

- d_1=*"apples taste good and are healthy"*
- d_2="*the taste of pears is delicious"*
- d_3="*basil adds a good taste to the dish"*

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	...	1	0
taste	1	0	...	2	1
...	...	...	...	...	...
good	1	2	...	0	0
pears	0	1	...	0	0

Co-occurrence matrix

- d_1="*apples taste good and are healthy*"
- d_2="*the taste of pears is delicious*"
- d_3="*basil adds a good taste to the dish*"

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	…	1	0
taste	1	0	…	2	1
...	…	…	…	…	…
good	1	2	…	0	0
pears	0	1	…	0	0

Co-occurrence matrix

- d_1="*apples taste good and are healthy*"
- d_2="*the taste of pears is delicious*"
- d_3="*basil adds a good taste to the dish*"

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	…	1	0
taste	1	0	…	2	1
...	…	…	…	…	…
good	1	2	…	0	0
pears	0	1	…	0	0

Co-occurrence matrix

- Embedding:
  - Basic: A word t is embedded to its row in the co-occurrence matrix.

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	…	1	0
taste	1	0	…	2	1
...	…	…	…	…	…
good	1	2	…	0	0
pears	0	1	…	0	0

Embedding of "apples"

- Embedding:
  - Basic: A word t is embedded to its row in the co-occurrence matrix.

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	…	1	0
taste	1	0	…	2	1
...	…	…	…	…	…
good	1	2	…	0	0
pears	0	1	…	0	0

Embedding of "pears"

- Embedding:
  - Basic: A word t is embedded to its row in the co-occurrence matrix.

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	...	1	0
taste	1	0	...	2	1
...	...	...	...	...	...
good	1	2	...	0	0
pears	0	1	...	0	0

Embedding of "pears"

- Intuition:
  - "apples" and "pears" have more similar rows than e.g. "apples" and "car".
    - Occur more often with "taste" than "car" does.

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

- Embedding:
  - Basic: A word t is embedded to its row in the co-occurrence matrix.

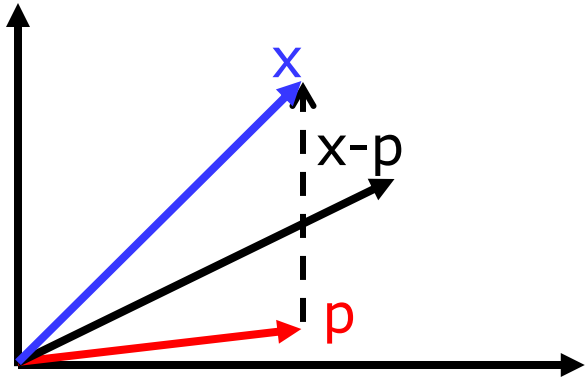
- Issue:
  - Very long vectors!
  - Solution: Dimensionality reduction.
    - One popular approach is based on PCA (Principal Component Analysis).

PCA

- Principal Component Analysis
  - Let $z_1,\ldots,z_k$ be k eigenvectors corresponding to largest eigenvalues of X^TX and project feature vector x to $p_x = \sum_{i=1}^{k}(x^T z_i)z_i$.
- Write x in the new basis:
  - $\hat{x} = \begin{pmatrix} x^T z_1 \\ \vdots \\ x^T z_k \end{pmatrix}$
- On whole data matrix:
  - X′=XZ with $Z = \begin{pmatrix} | & \ldots & | \\ z_1 & \ldots & z_k \\ | & \ldots & | \end{pmatrix}$



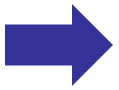
MNIST



Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Matrix M with entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.
  - Do eigendecomposition and replace M by MZ to get dxk matrix.
  - Use rows as embeddings of words.

	apples	taste	...	good	pears
apples	0	1	...	1	0
taste	1	0	...	2	1
...	...	...	...	...	...
good	1	2	...	0	0
pears	0	1	...	0	0



apples	0.1	...	-0.1
taste	1.0	...	-1.3
...	...	...	...
good	0.7	...	0.3
pears	0.2	...	0.0

- Main Issues:
  - Slow to compute eigendecomposition on large matrices.
  - Memory heavy to compute eigendecomposition.

CBOW: Continuous Bag of Words

Machine Learning on Text

- Spam Filter:

Greetings,

I seek for a trusted, sincere and honest person to judiciously set up a charity foundation under my sponsorship.

The foundation will be strictly for helping the poorest of the poor; especially the orphanage homes and homes for destitute and the mentally retarded.

I am 76 years old and suffering from Protracted Cancer Of The Lungs and on admission at the specialist hospital intensive care unit.

My lawyer that was supposed to handle the process of executing my Will & Testament was shot during the protest against the presidential election.

If t touched your spirit to be part in putting smile and new life on the vulnerable, kindly reply through my personal below email for more information.

Mrs. Matsushita

Email: matsumorand@gmail.com

Formulaire sans titre

REMPLIR LE FORMULAIRE

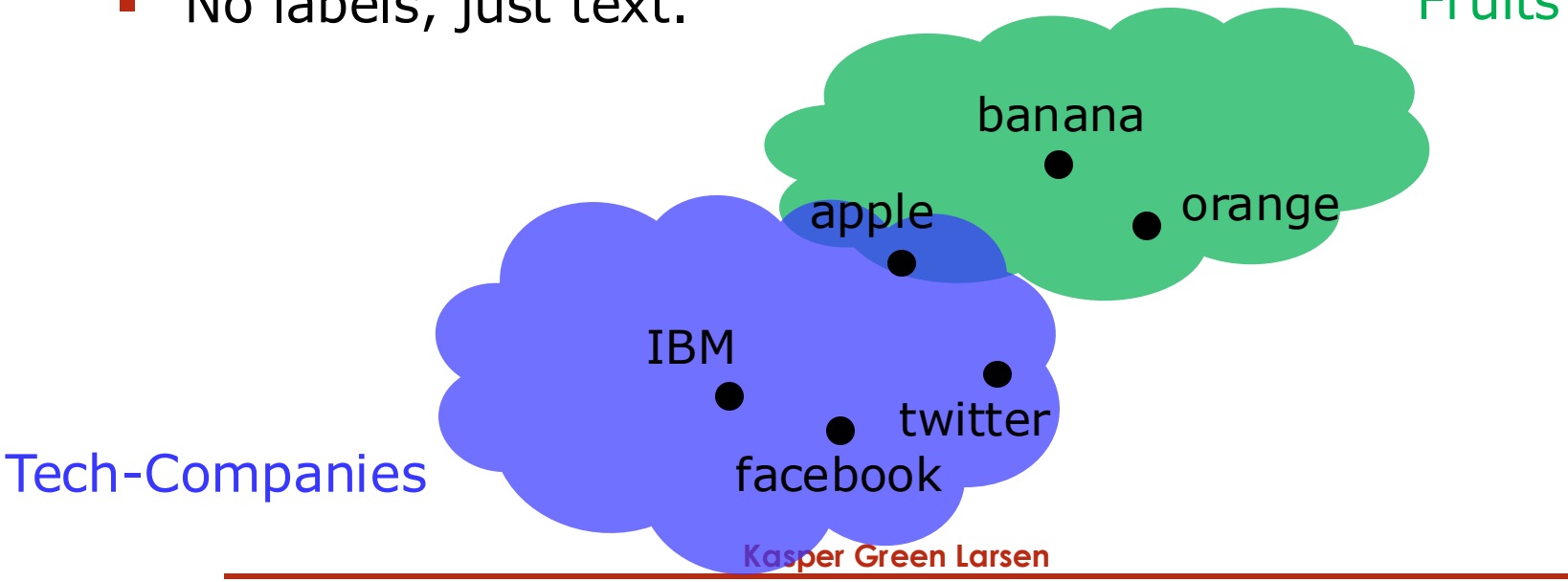
Word Embeddings

- Bag-of-words:
  - Each word t corresponds to a standard unit vector $w(t)=e_t$.
  - Document d represented as vector v(d):
    - $v(d) = \sum_{t \in d} w(t)$
- Issue:
  - Similarity between words ignored.
  - Would be sensible if v(d) was similar to v(d') if replace all words in d with synonyms to obtain d'.
- Question:
  - How can we define word vectors that "preserve" "similarity" between words?

Word Embeddings

- Question:
  - How can we define word vectors that ”preserve” ”similarity” between words?

- Unsupervised Learning!
  - Learn similarities between words from data.
  - No labels, just text.



Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.
  - Do eigendecomposition and replace M by MZ to get dxk matrix.
  - Use rows as embeddings of words.

	apples	taste	...	good	pears
apples	0	1	...	1	0
taste	1	0	...	2	1
...	...	...	...	...	...
good	1	2	...	0	0
pears	0	1	...	0	0

- Main Issues:
  - Slow to compute eigendecomposition on large matrices.
  - Memory heavy to compute eigendecomposition.

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.
  - Do eigendecomposition and replace M by MZ to get dxk matrix.
  - Use rows as embeddings of words.

	apples	taste	...	good	pears
apples	0	1	...	1	0
taste	1	0	...	2	1
...	...	...	...	...	...
good	1	2	...	0	0
pears	0	1	...	0	0

- Another Issues:
  - Hand-crafted features by human experts.
  - Lesson: ML algorithms often better than human experts.

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.
  - Do eigendecomposition and replace M by MZ to get dxk matrix.
  - Use rows as embeddings of words.

	apples	taste	...	good	pears
apples	0	1	...	1	0
taste	1	0	...	2	1
...	...	...	...	...	...
good	1	2	...	0	0
pears	0	1	...	0	0

- Idea:
  - Use learning to find good embeddings of words!

CBOW

- Defining a Supervised Learning Problem

- Context window: Specified by a width around a wor
  - Example: Context window of width 2 around ”w

context window

- *”partner abroad who can accommodate 200M USD for Investment”*

- Learning Task:
  - Predict middle word from context.
  - Example:
    - Input is (”partner”,”abroad”,”can”,”accommodate”)
    - Label is ”who”

Bootstrap issue: Need a representation to learn a repr.?

How do we represent these as input to a learning algo?

CBOW

- Defining a Supervised Learning Problem
  - Example:
    - Input is (”partner”,”abroad”,”can”,”accommodate”)
    - Label is ”who”

- Input Format:
  - Use ”naive” one-hot encoding from bag-of-words.
  - Word t represented by standard unit vector e_t.

$e_{\text{”for”}}$ (0,1,0,0,…,…..,0,0,0,0,….,0,0,……0,0,0,……,0,0,0,…….)

$e_{\text{”can”}}$ (0,0,0,0,…,…..,0,1,0,0,….,0,0,……0,0,0,……,0,0,0,…….)

$e_{\text{”who”}}$ (0,0,0,0,…,…..,0,0,0,0,….,1,0,……0,0,0,……,0,0,0,…….)

CBOW

”partner abroad who can accommodate 200M USD for Investment”

- Continuous Bag of Words
  - Example:
    - Input is (”partner”,”abroad”,”can”,”accommodate”)
    - Label is ”who”

$e_{\text{”partner”}}$ (0,1,0,0,…,…..,0,0,0,0,….,0,0,……0,0,0,…….,0,0,0,…….)

$e_{\text{”abroad”}}$ (0,0,0,0,…,…..,0,1,0,0,….,0,0,……0,0,0,…….,0,0,0,…….)

$e_{\text{”can”}}$ (0,0,0,0,…,…..,0,0,0,0,….,1,0,……0,0,0,…….,0,0,0,…….)

$e_{\text{”accommodate”}}$ (0,0,0,0,…,…..,0,0,0,0,….,0,0,……0,1,0,…….,0,0,0,…….)

$e_{\text{”who”}}$ (0,0,0,0,…,…..,0,0,0,0,….,0,0,……0,0,0,…….,0,1,0,…….)

CBOW

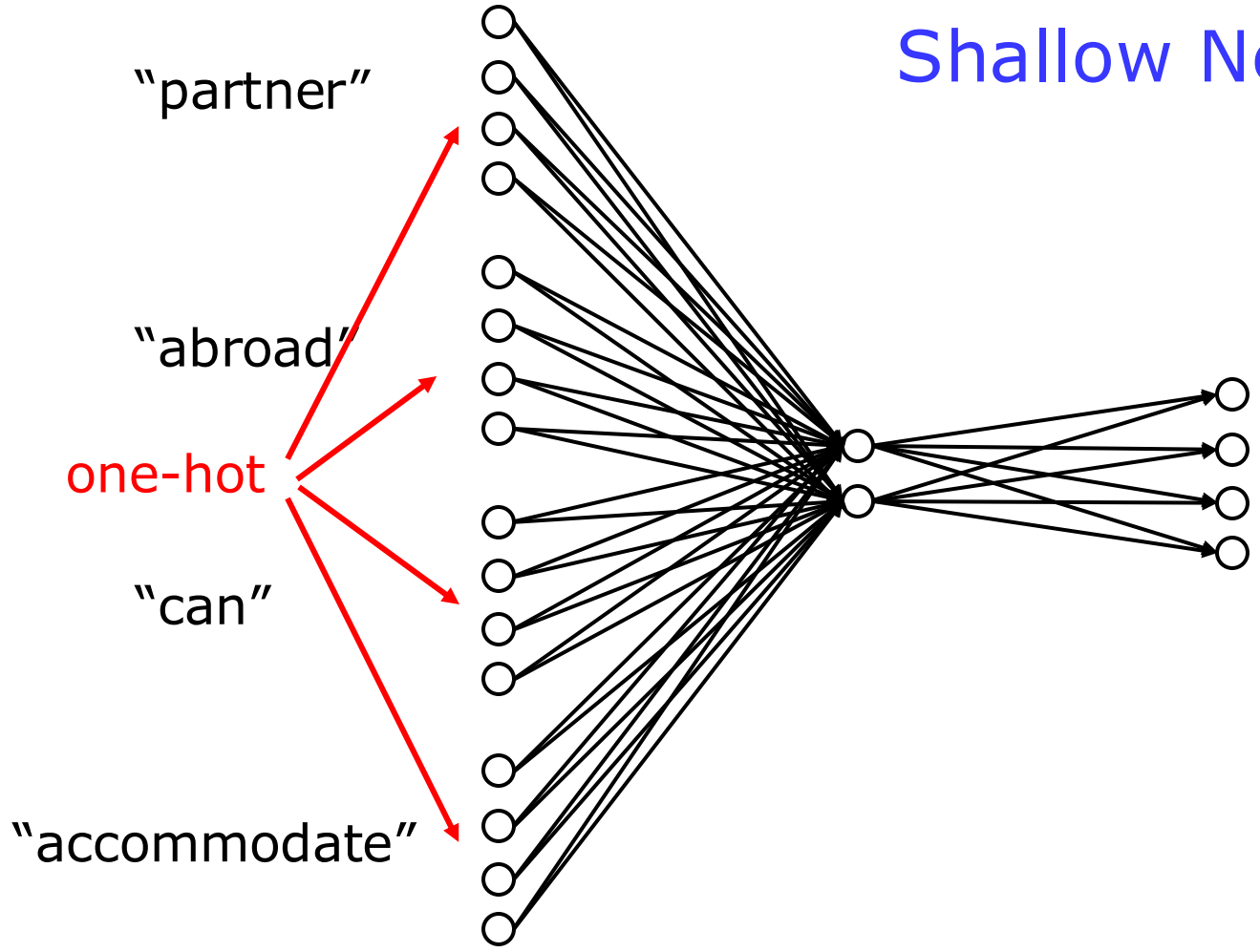
”partner abroad who can accommodate 200M USD for Investment”

- Continuous Bag of Words
  - From documents $d_1,..,d_n$ generate training data:
    - Construct dictionary with all the words that occur.
    - For each document and each word t in document:
      - Generate training example with context as input features and t as target label.
      - All one-hot encoded as indices into dictionary.

- Example:
  - ((”partner”,”abroad”,”can”,”accommodate”),”who”)
  - ((”abroad”,”who”,”accommodate”,”200M”),”can”)
  - …
  - ((”taste”,”of”,”is”,”delicious”),”pears”)

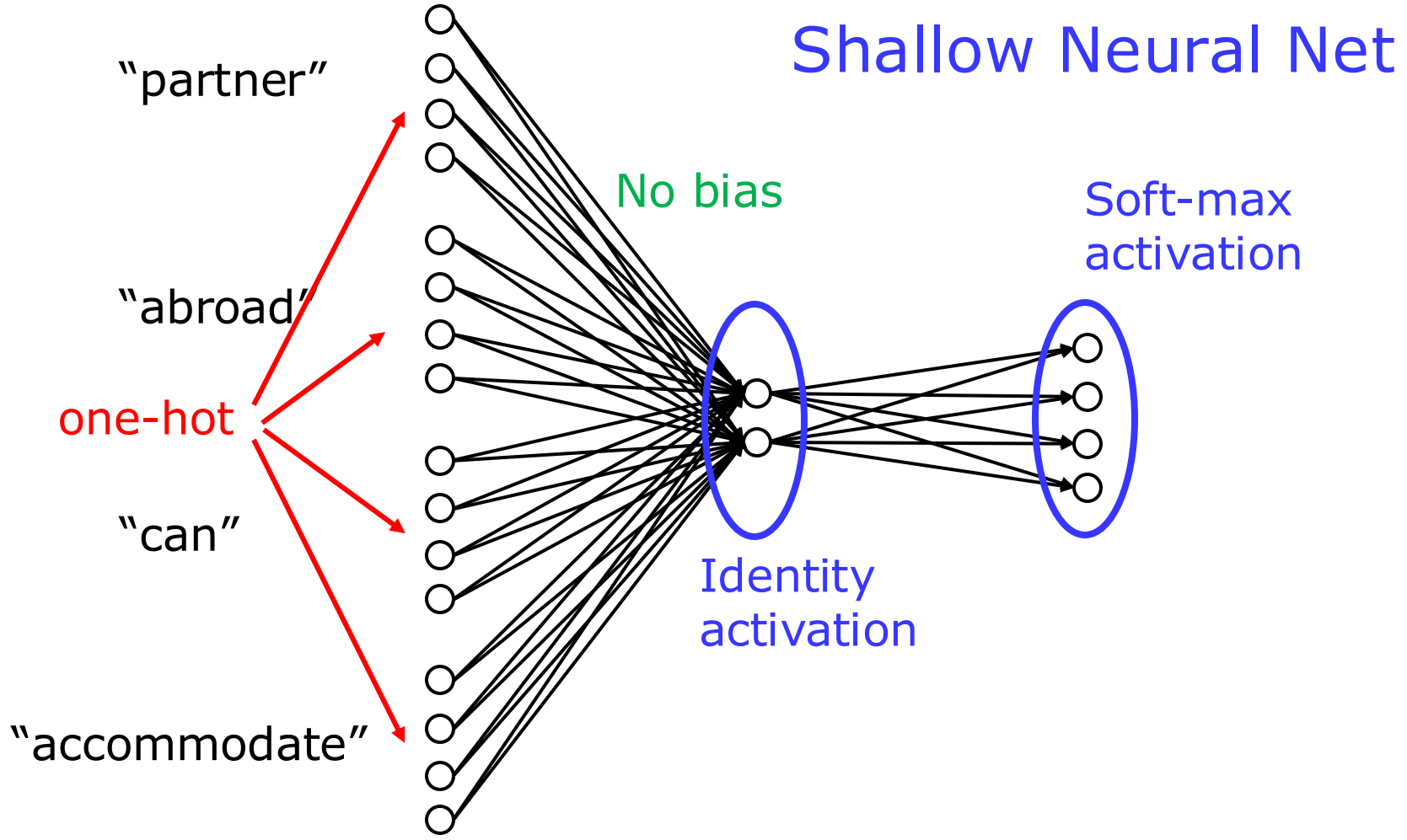
CBOW

((”partner”,”abroad”,”can”,”accommodate”),”who”)

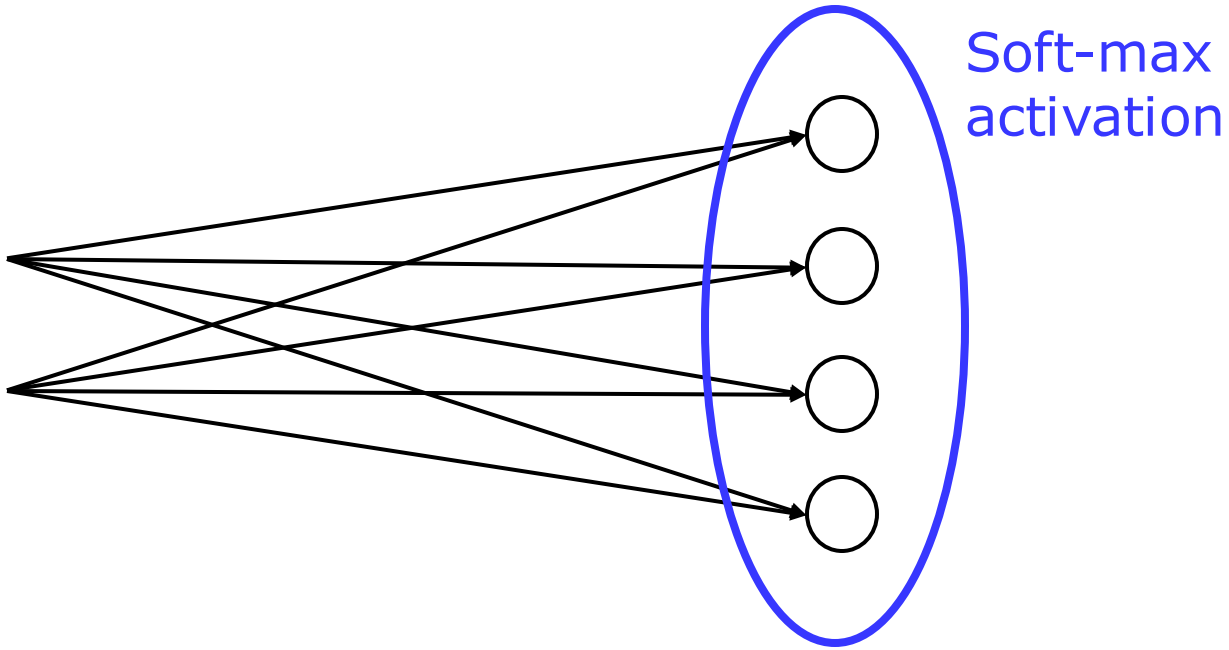


CBOW

((”partner”,”abroad”,”can”,”accommodate”),”who”)



CBOW ((”partner”,”abroad”,”can”,”accommodate”),”who”)



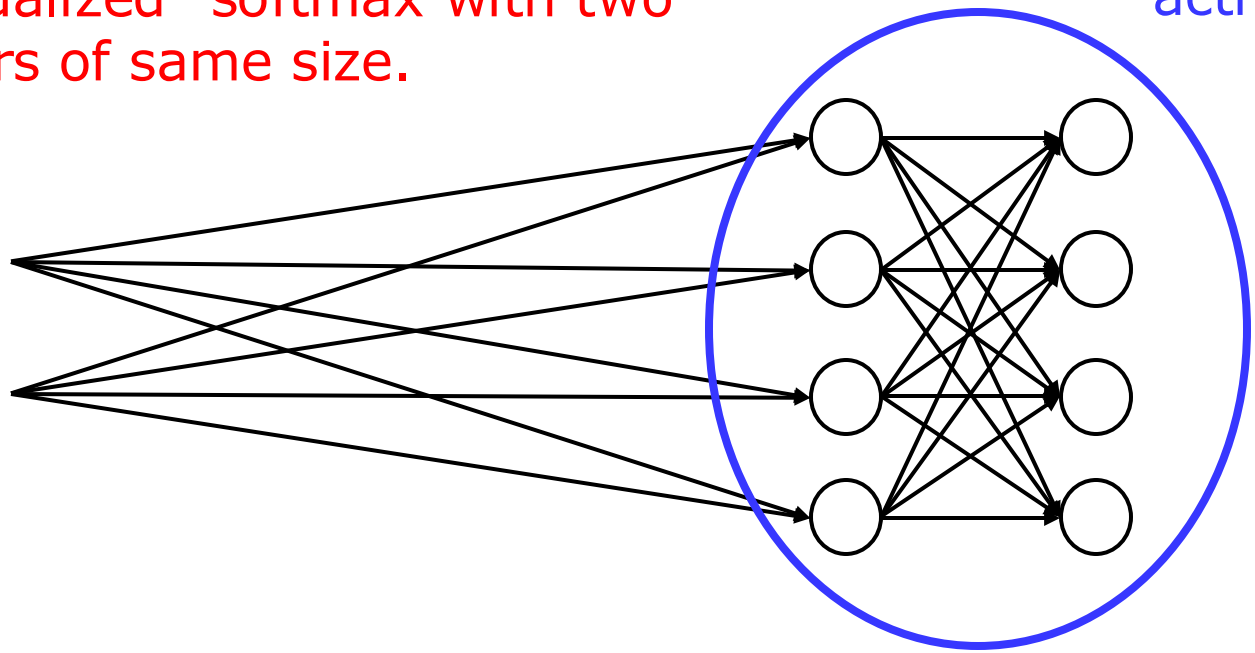
Output: Non-negative summing to 1.

Intuition: Entry t is probability of word t given the context.

CBOW ((”partner”,”abroad”,”can”,”accommodate”),”who”)

In neural net lectures, we “visualized” softmax with two layers of same size.

Soft-max activation

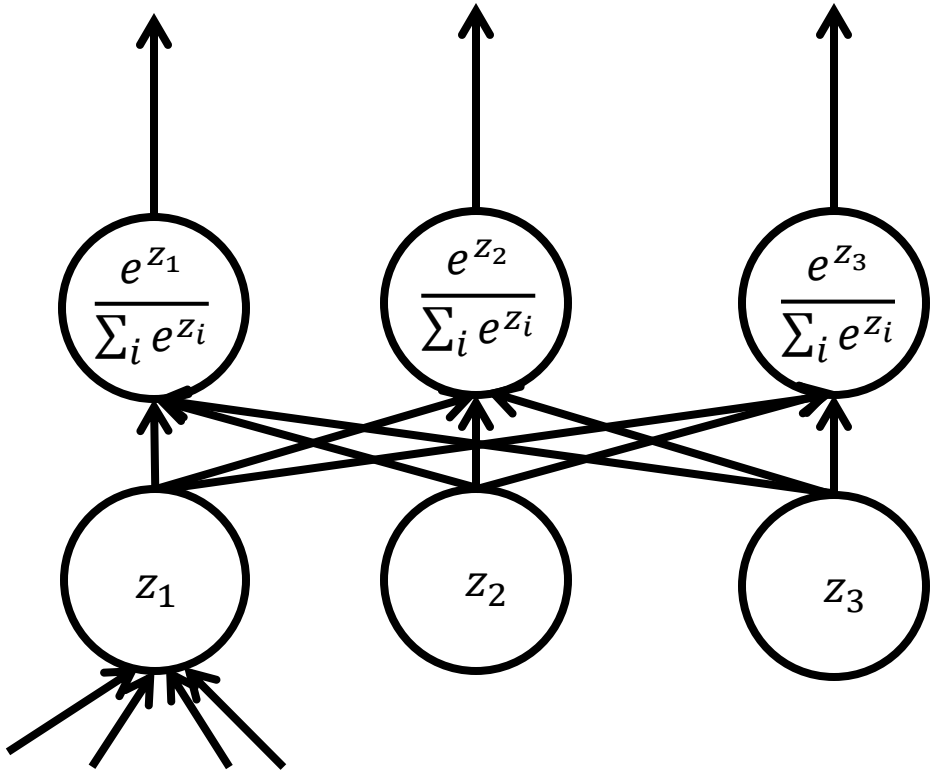


Output: Non-negative summing to 1.

Intuition: Entry t is probability of word t given the context.

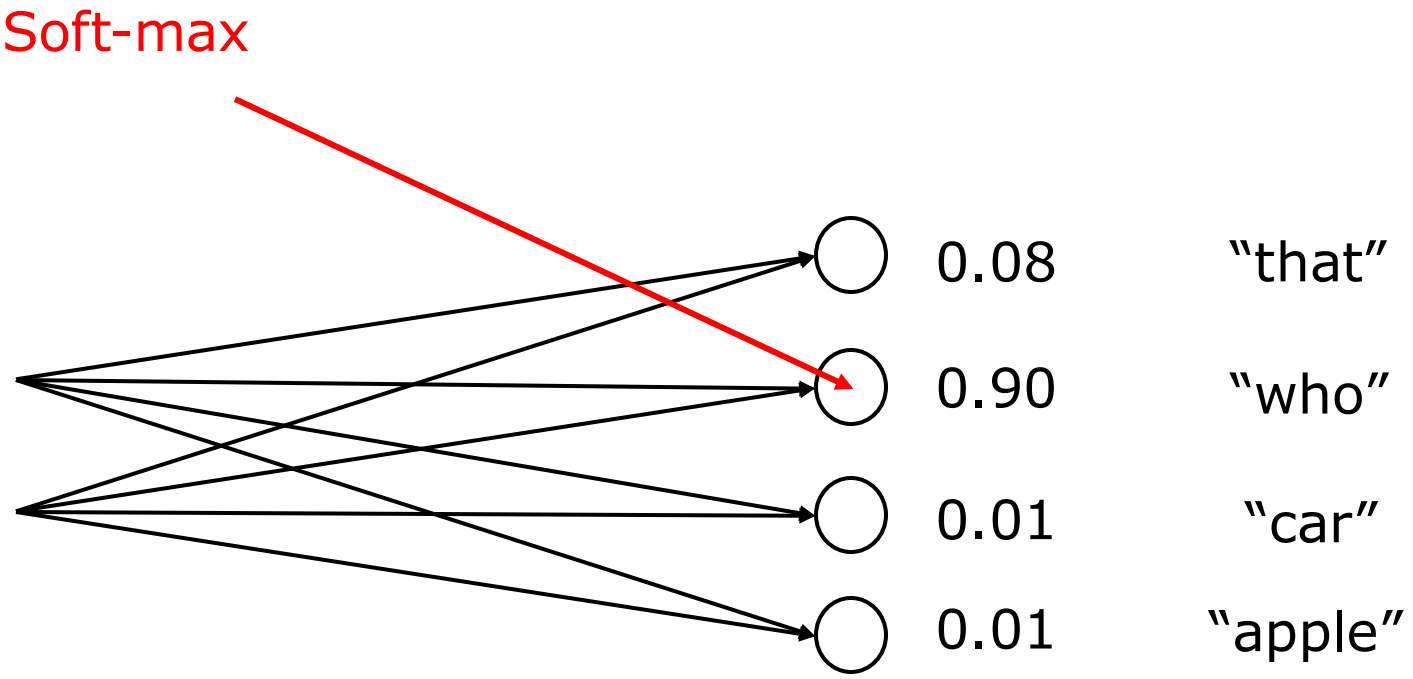
Activation Functions

- Multiclass Classification Output Neurons
  - For classification problems with negative log-likelihood (like multinomial/cross-entropy/softmax regression), often use Softmax neurons $\mathrm{softmax}(z)_i$.



CBOW

((”partner”,”abroad”,”can”,”accommodate”),”who”)



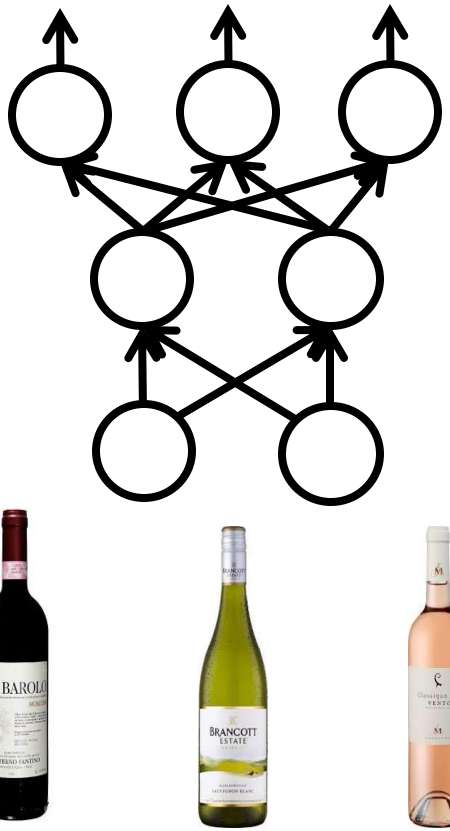
Output: Non-negative summing to 1.

Intuition: Entry t is probability of word t given the context.

Cross Entropy Loss

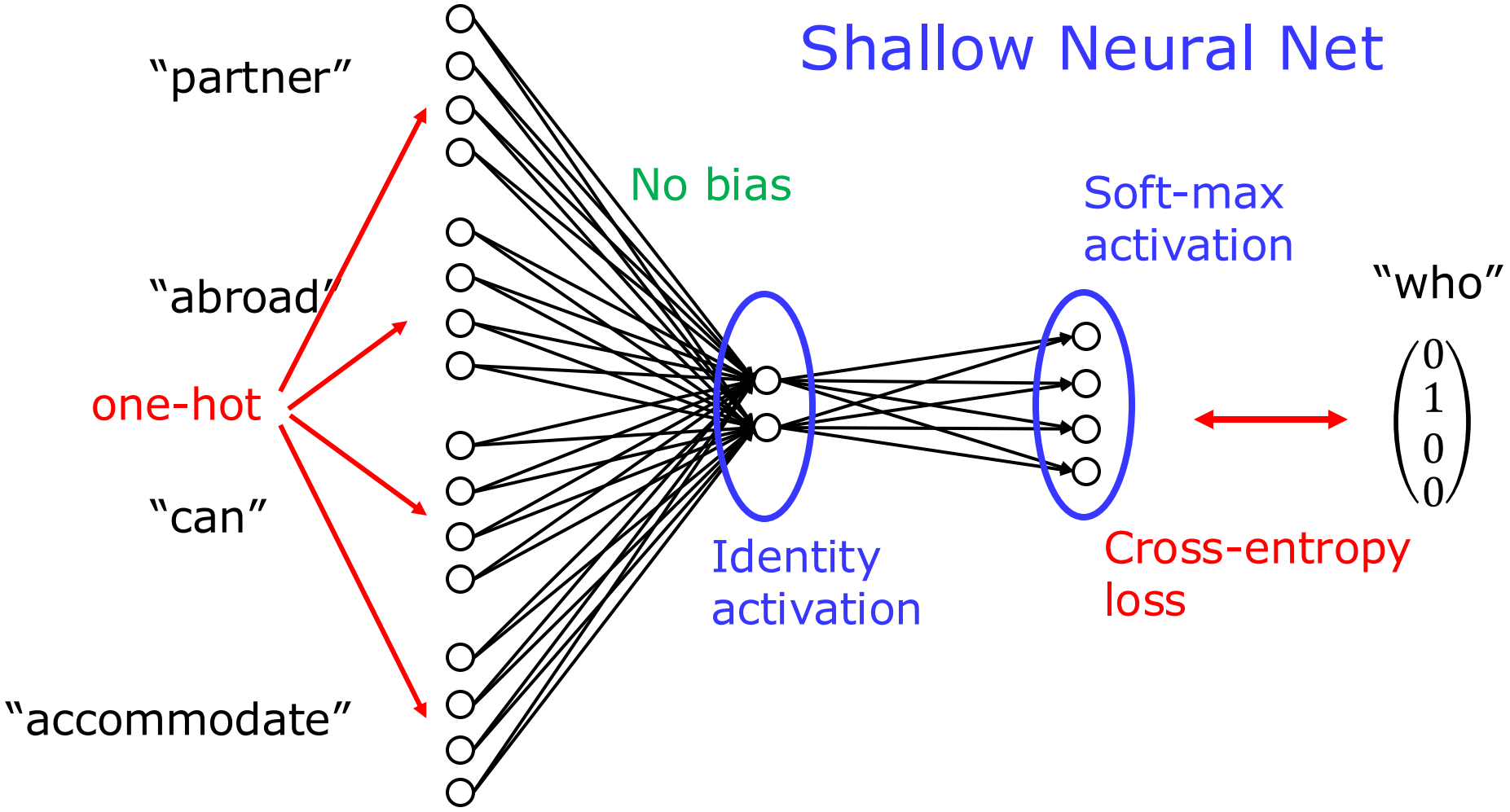
Our setup:
y is one-hot.

- Loss function for multiclass classification:
  - $L(h(x), y) = -\sum_{j=1}^{d} y_j \ln(h(x)_j)$
  - $E_{in}(h) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{d} y_{i,j} \ln(h(x_i)_j)$
- Setup:
  - Any machine learning model h outputting d probabilities $p_1,\ldots,p_d$ summing to 1.
    - $p_j = h(x)_j$
  - Intention $\Pr[y=j|x] = p_j$.
- Labels y:
  - Vectors of length d. Non-negative entries summing to 1 (i.e. probability distribution).



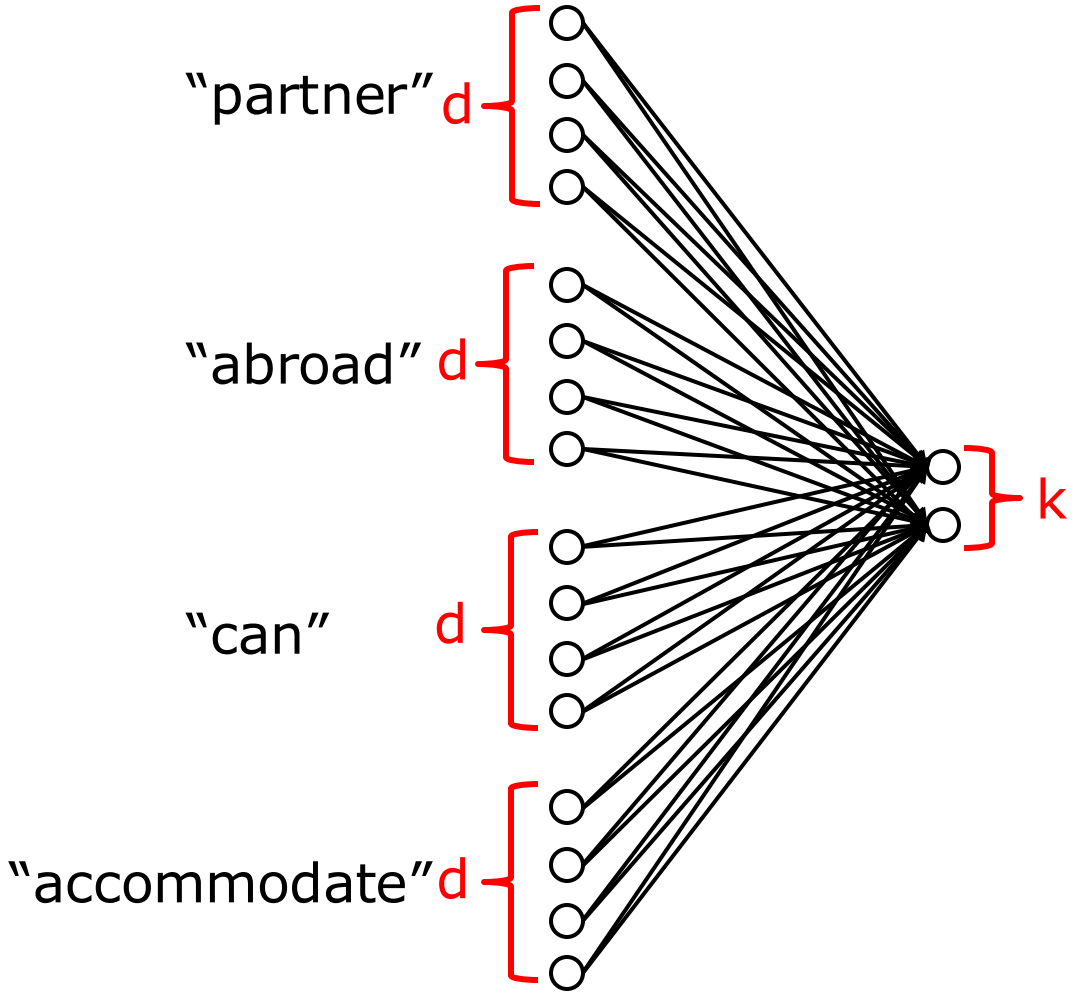
CBOW

((”partner”,”abroad”,”can”,”accommodate”),”who”)



CBOW

((”partner”,”abroad”,”can”,”accommodate”),”who”)



Intuition:

Want middle layer to give the embedding of a word.

Similar to auto-encoder.

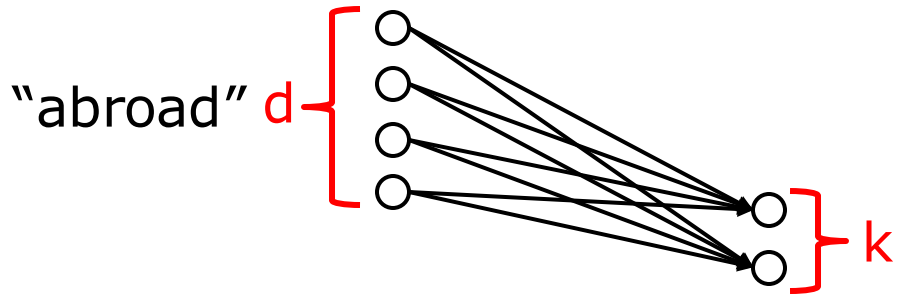
Issue:

Takes 4 words in.

Need a way to get embedding of a single word.

CBOW

((”partner”,”abroad”,”can”,”accommodate”),”who”)



Weights:

Represented by dxk matrix W.

One-hot:

Just extracts row of W.

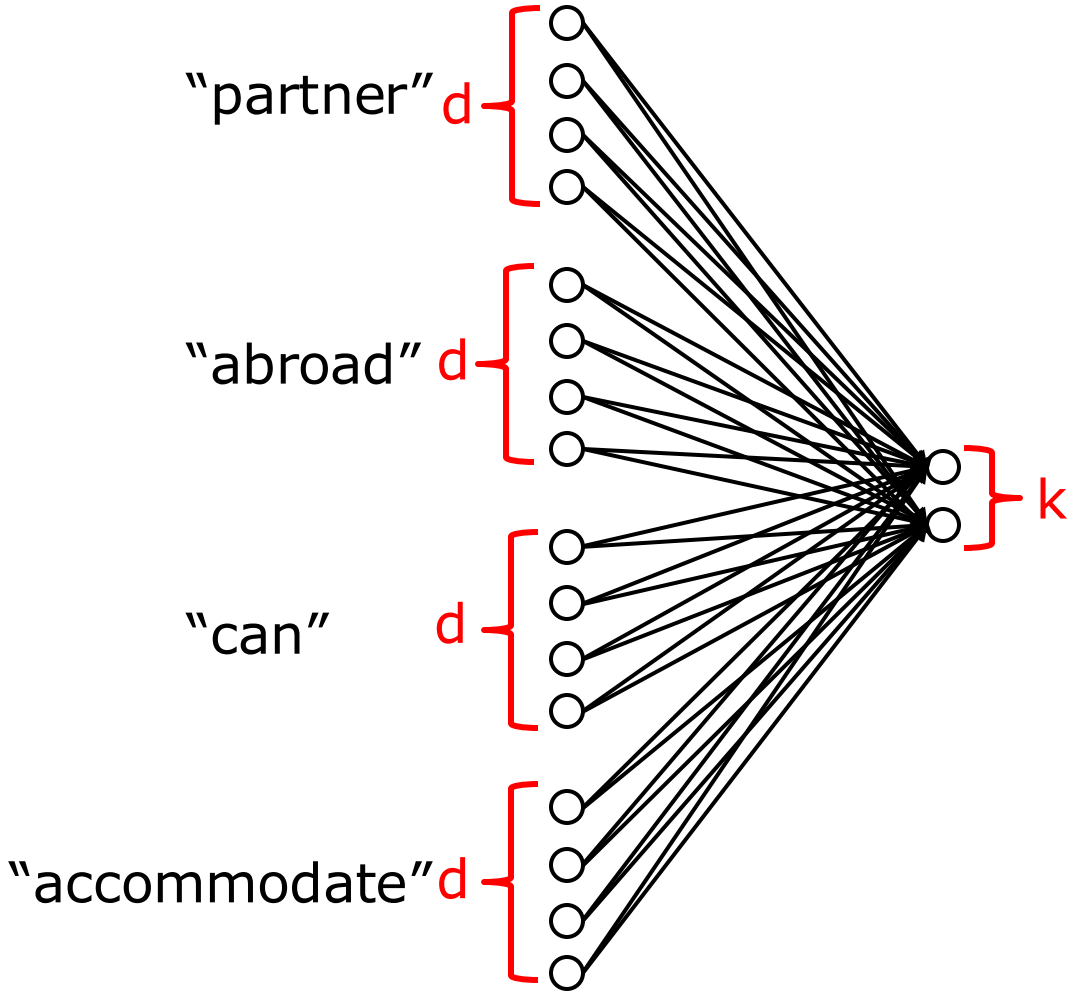
$$e_{"abroad"} W = \begin{pmatrix} 1 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} a & b \\ c & d \\ e & f \\ g & h \end{pmatrix} = \begin{pmatrix} a & b \end{pmatrix}$$

Intuition:

Rows of W gives embeddings of words.

CBOW

((”partner”,”abroad”,”can”,”accommodate”),”who”)



Issue:

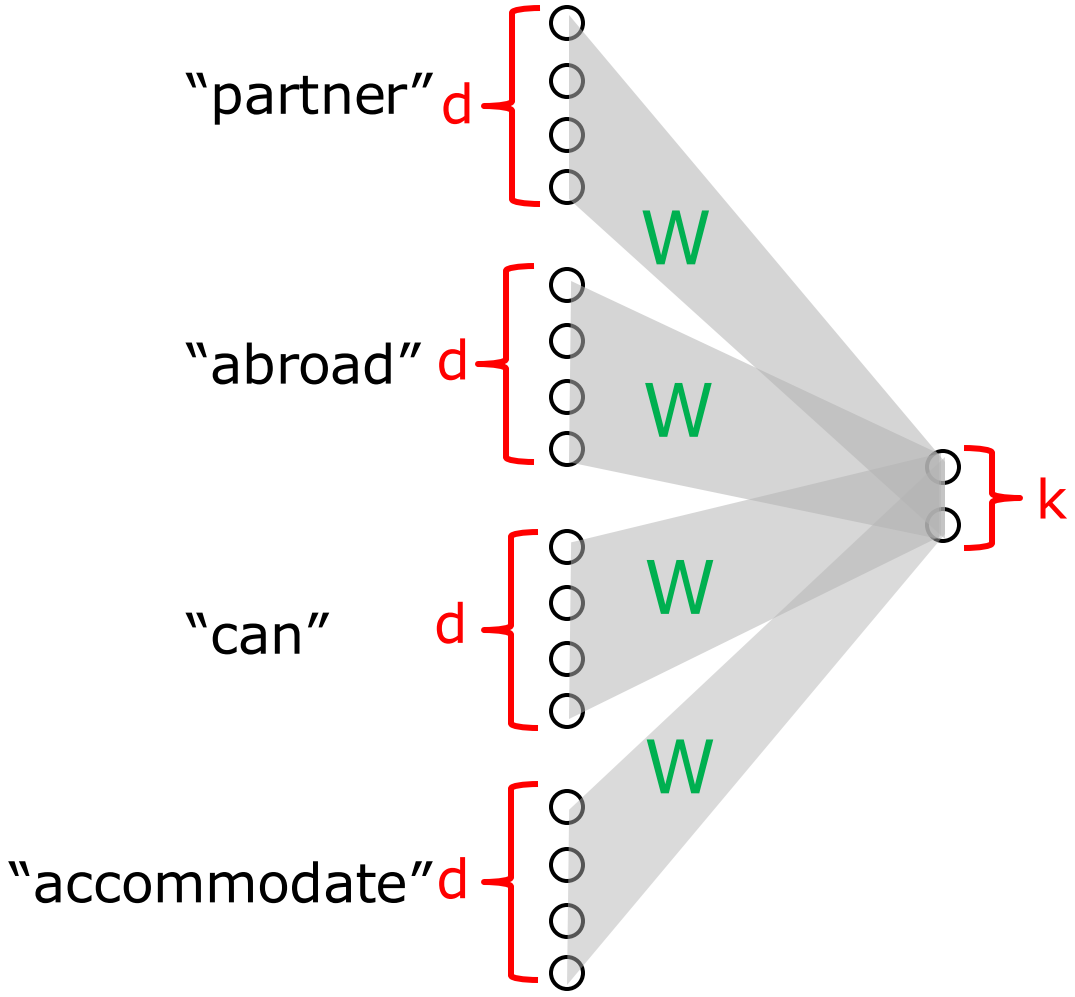
Have 4 different representations of all the words.

Solution:

Force same weights on the four inputs.

CBOW

((”partner”,”abroad”,”can”,”accommodate”),”who”)



Issue:

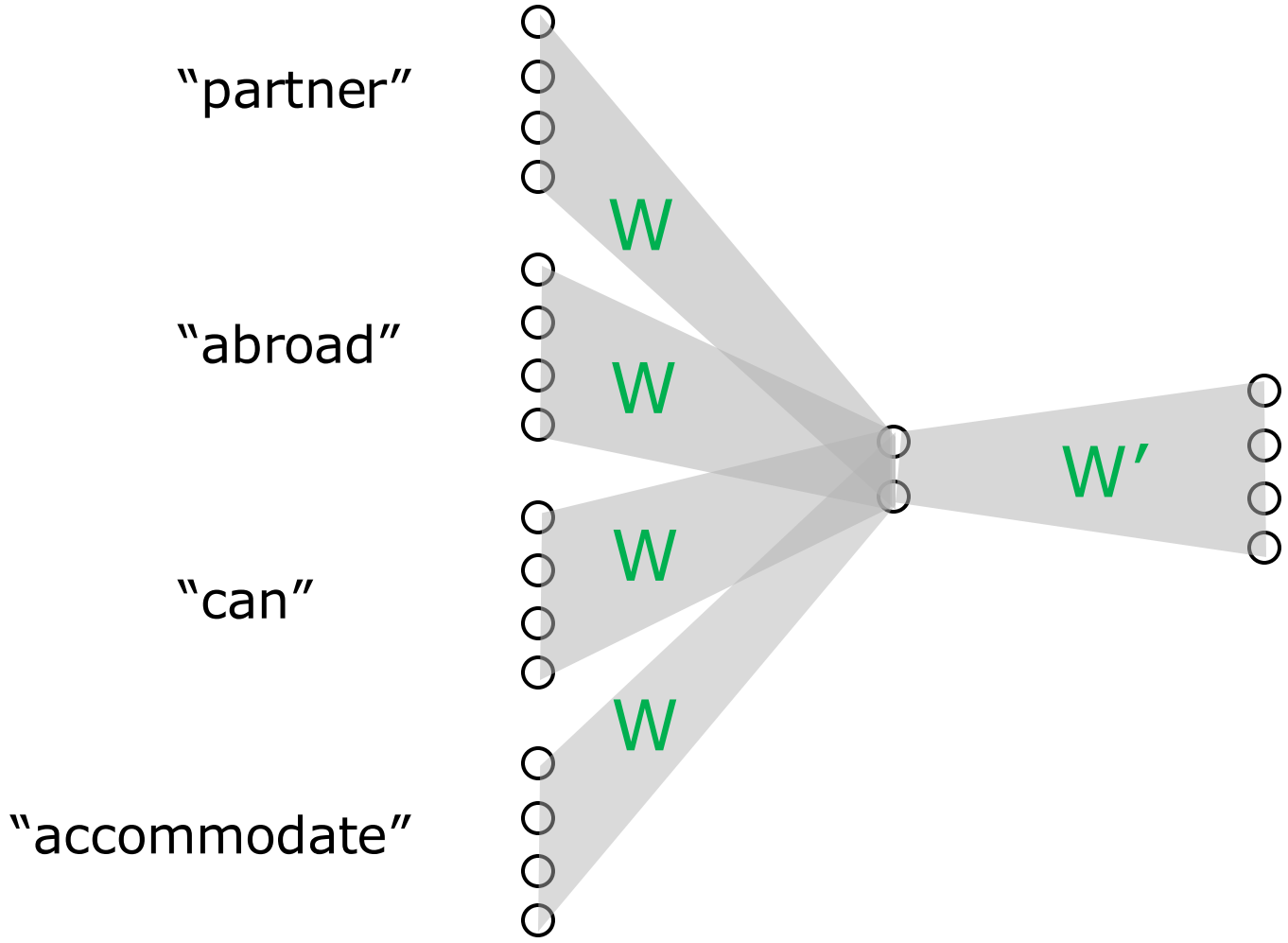
Have 4 different representations of all the words.

Solution:

Force same weights on the four inputs.

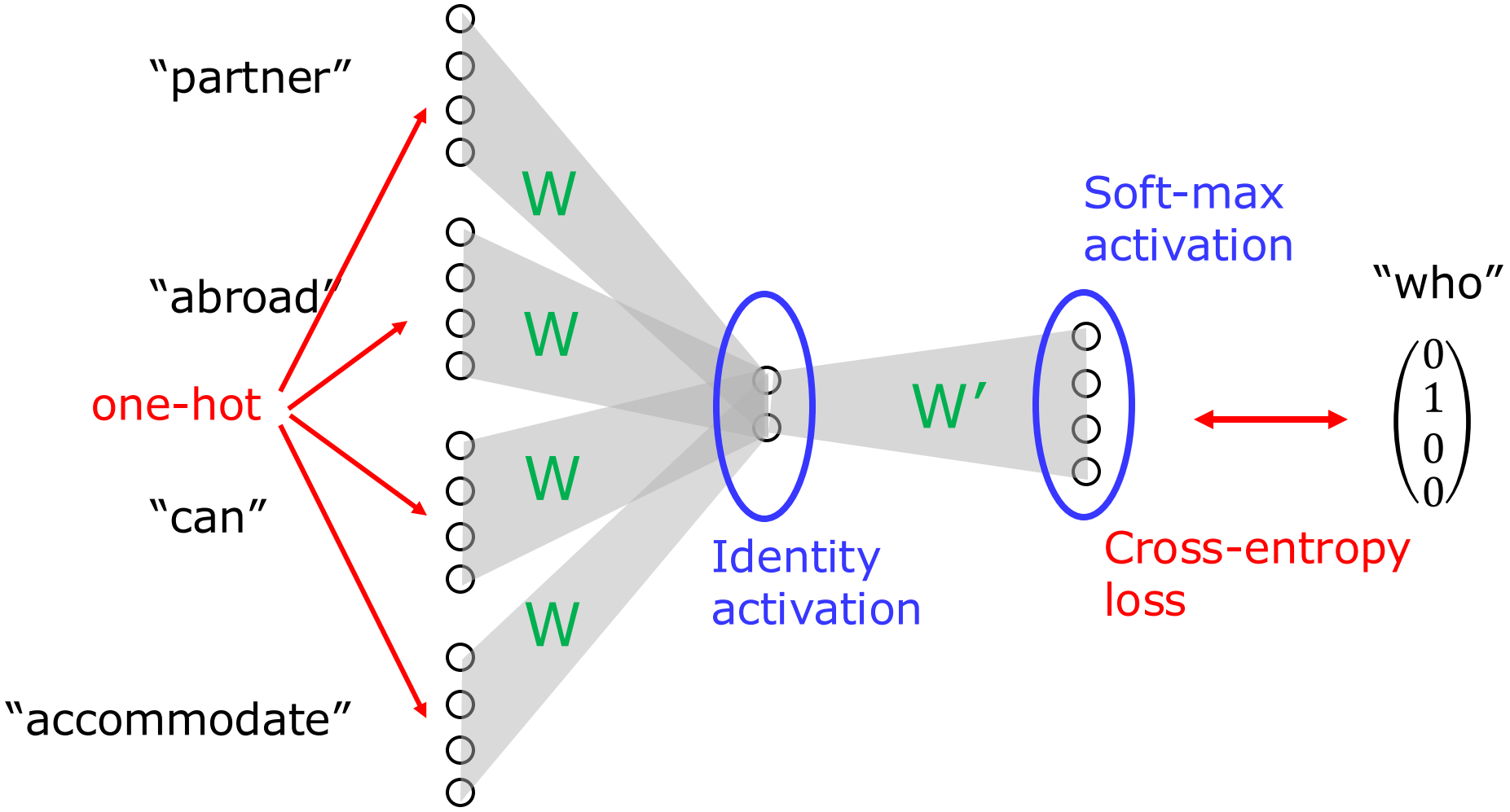
CBOW

((”partner”,”abroad”,”can”,”accommodate”),”who”)



CBOW

((”partner”,”abroad”,”can”,”accommodate”),”who”)



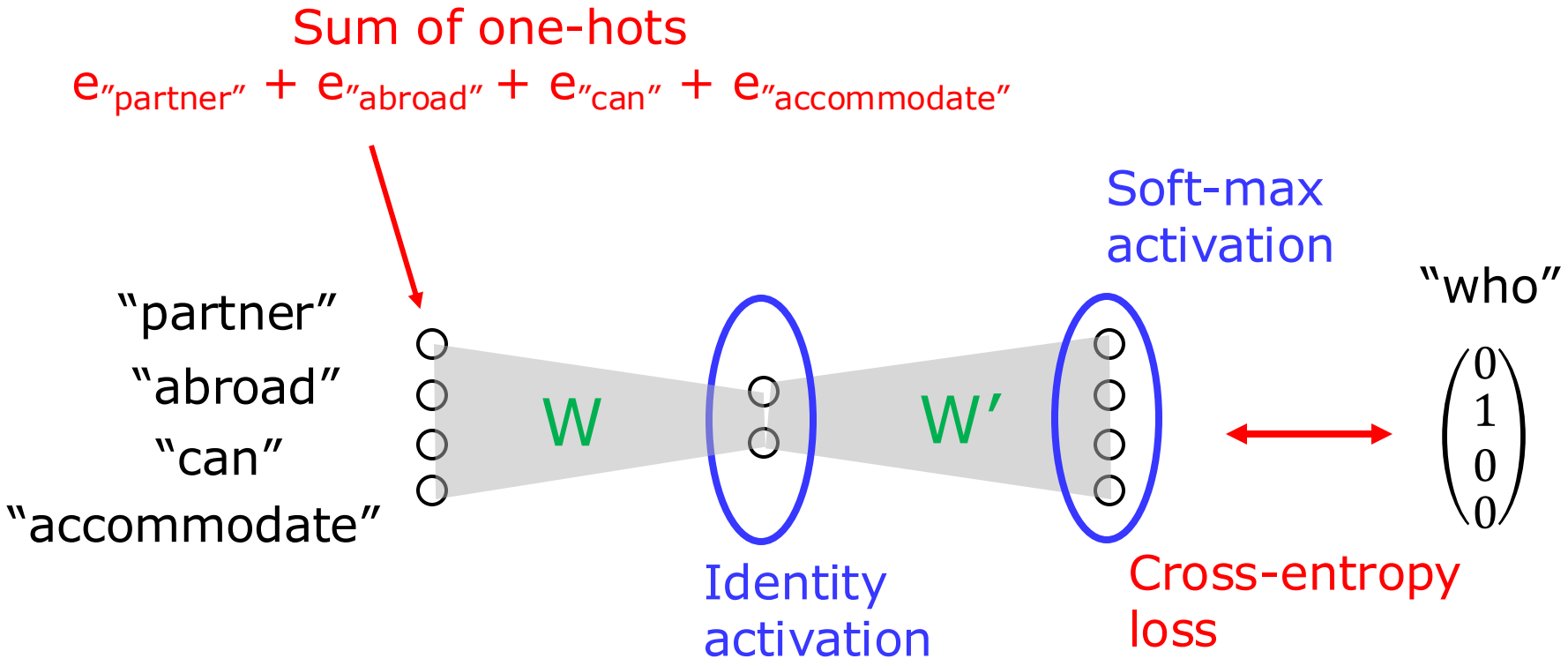
CBOW ((”partner”,”abroad”,”can”,”accommodate”),”who”)

- Observation:
  - Middle layer output equals:
    - $e_{\text{”partner”}}W+e_{\text{”abroad”}}W+e_{\text{”can”}}W+e_{\text{”accommodate”}}W$
  - Same as:
    - $(e_{\text{”partner”}} + e_{\text{”abroad”}} + e_{\text{”can”}} + e_{\text{”accommodate”}})W$

- Training Data:
  - Sum the one-hot encodings of the context words and use that as input feature vector:
    - $((e_{\text{”partner”}} + e_{\text{”abroad”}} + e_{\text{”can”}} + e_{\text{”accommodate”}}),e_{\text{”who”}})$

CBOW

((”partner”,”abroad”,”can”,”accommodate”),”who”)

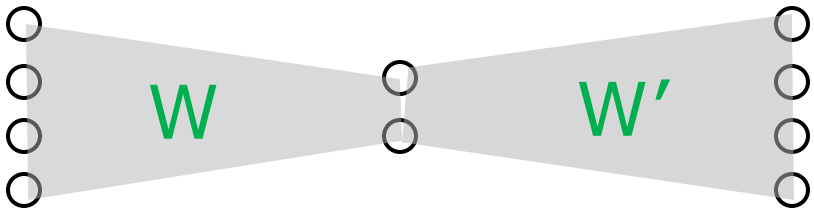


Train on all the training documents $d_1,\ldots,d_n$.

Mini-batch Stochastic Gradient Descent as always.

CBOW

- Obtaining a word's embedding:
  - To obtain embedding of t:
    - $e_t W$
  - Same as:
    - t'th row of W



W=

apples	0.1	-1.2	…	1.4	-0.1
taste	1.0	0.2	…	2.1	-1.3
…	…	…	…	…	…
good	0.7	2.1	…	0.5	0.3
pears	0.2	-1.0	…	1.3	0.0

Co-Occurrence Matrices

Machine Learning on Text

- Spam Filter:

Greetings,

I seek for a trusted, sincere and honest person to judiciously set up a charity foundation under my sponsorship.

The foundation will be strictly for helping the poorest of the poor; especially the orphanage homes and homes for destitute and the mentally retarded.

I am 76 years old and suffering from Protracted Cancer Of The Lungs and on admission at the specialist hospital intensive care unit.

My lawyer that was supposed to handle the process of executing my Will & Testament was shot during the protest against the presidential election.

If t touched your spirit to be part in putting smile and new life on the vulnerable, kindly reply through my personal below email for more information.

Mrs. Matsushita

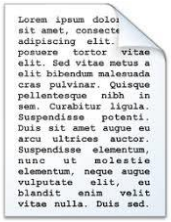
Email: matsumorand@gmail.com

Formulaire sans titre

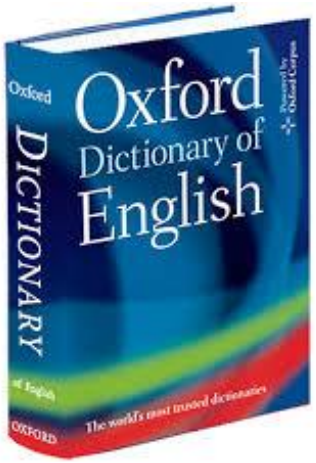
REMPLIR LE FORMULAIRE

Bag of Words

- Goal: Represent text as a vector.

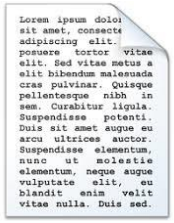


"Viagra Oral Jelly is a remedy that helps to control and restore potency during sexual arousal. The main advantage of Viagra Oral Jelly is its fast action"



- Bag of Words:
  - Assume all words in text come from the English dictionary.

Vector has 1 entry per word in dictionary





"Viagra"

(0,1,0,0,…,2,0,1,0)

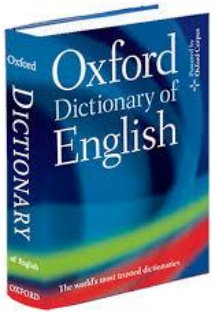
"potency" "cow"

Entry stores count of many times word occurs in text.

TF-IDF



- Bag-of-words:
  - Entry t in vector for document d is:
    - $f_{t,d}$ = #occurrences of t in d
- Tf-idf:
  - Entry t in vector for document d is:
    - $tf(t,d) \cdot idf(t) = \frac{f_{t,d}}{|d|} \cdot \ln\left(\frac{n}{|\{d_i : t \in d_i\}|}\right)$



- Issue:
  - Similarity between words ignored:
    - ”man”, ”boy”,”woman”,”girl” not treated differently from ”apple”,”football” and ”this”.

Word Embeddings



- Bag-of-words:
  - Entry t in vector for document d is:
    - $f_{t,d}$ = #occurrences of t in d

- Formalizing the issue:
  - Think of each word as being represented by a vector.
  - In Bag-of-words:
    - Each word t corresponds to a standard unit vector $w(t)=e_t$.
    - Document d represented as vector v(d):
      - $v(d) = \sum_{t \in d} w(t)$

Word Embeddings



- Bag-of-words:
  - Each word t corresponds to a standard unit vector $w(t)=e_t$.
  - Document d represented as vector v(d):
    - $v(d) = \sum_{t \in d} w(t)$
- *"partner abroad who can accommodate 200M USD for Investment"*

"for" "can" "who" "abroad" "partner"

v(d) (0,1,0,0,…,…..,0,1,0,0,….,1,0,……0,1,0,…….,0,0,1,…….)

$e_{\text{"for"}}$ (0,1,0,0,…,…..,0,0,0,0,….,0,0,……0,0,0,…….,0,0,0,…….)

$e_{\text{"can"}}$ (0,0,0,0,…,…..,0,1,0,0,….,0,0,……0,0,0,…….,0,0,0,…….)

$e_{\text{"who"}}$ (0,0,0,0,…,…..,0,0,0,0,….,1,0,……0,0,0,…….,0,0,0,…….)

Word Embeddings

- Bag-of-words:
  - Each word t corresponds to a standard unit vector w(t)=e_t.
  - Document d represented as vector v(d):
    - $v(d) = \sum_{t \in d} w(t)$
- Issue:
  - Similarity between words ignored:
    - ”man”, ”boy”,”woman”,”girl” not treated differently from ”apple”,”football” and ”this”.
    - $w(\text{"man"})^T w(\text{"boy"}) = 0$
    - $w(\text{"man"})^T w(\text{"this"}) = 0$
    - $\|w(\text{"woman"}) - w(\text{"girl"})\| = \|w(\text{"boy"}) - w(\text{"apple"})\|$

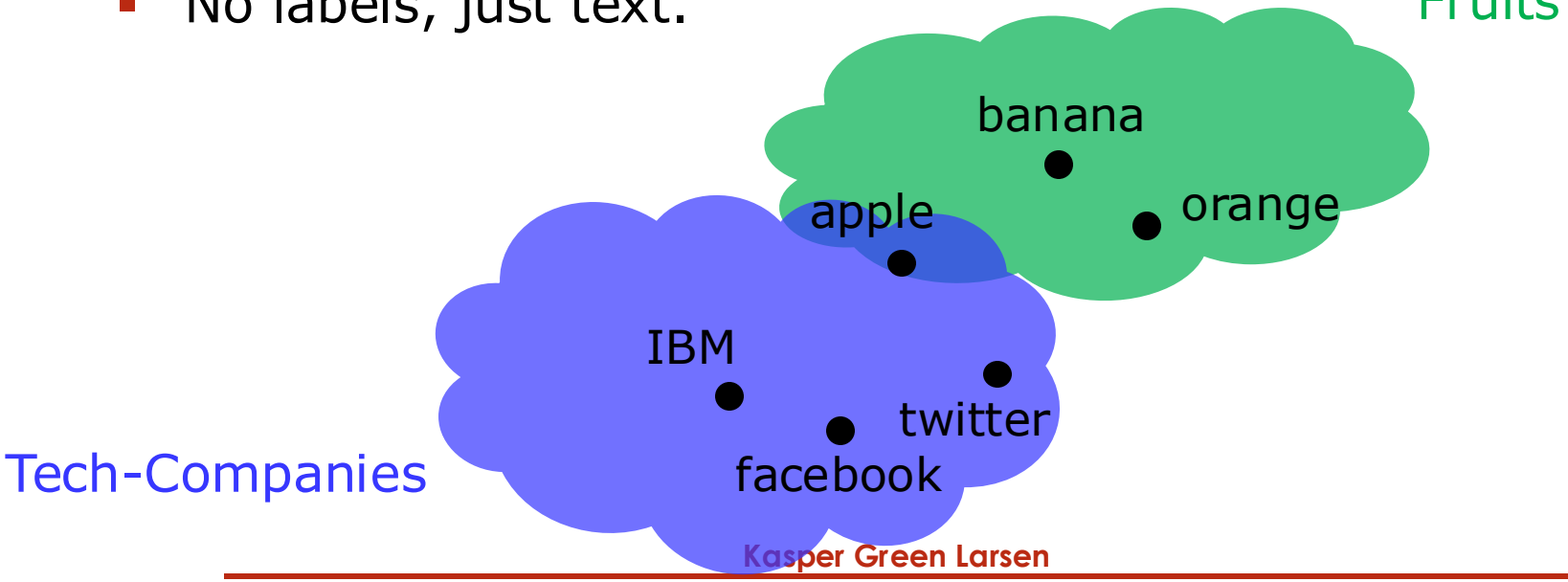
Word Embeddings

- Bag-of-words:
  - Each word t corresponds to a standard unit vector $w(t)=e_t$.
  - Document d represented as vector v(d):
    - $v(d) = \sum_{t \in d} w(t)$
- Issue:
  - Similarity between words ignored.
  - Would be sensible if v(d) was similar to v(d') if replace all words in d with synonyms to obtain d'.
- Question:
  - How can we define word vectors that "preserve" "similarity" between words?

Word Embeddings

- Question:
  - How can we define word vectors that ”preserve” ”similarity” between words?

- Unsupervised Learning!
  - Learn similarities between words from data.
  - No labels, just text.



Word Embeddings

- Co-Occurrence Matrix:
  - Base similarity on occurring in similar sentence structures / together with similar words.

Co-occurrences:
("who","partner")
("who","abroad")
("who","can")
("who","accommodate")

- Two concepts: Co-occurrence and context window.

context window

- *"partner abroad who can accommodate 200M USD for Investment"*

- Context window: Specified by a width around a word.
  - Example: Context window of width 2 around "who".
- Co-Occurrence: Have a co-occurrence between "who" and every word in the context window around "who".

Word Embeddings

- Co-Occurrence Matrix:
  - Base similarity on occurring in similar sentence structures / together with similar words.

Co-occurrences:
("can","abroad")
("can","who")
("can","accommodate")
("can","200M")

- Two concepts: Co-occurrence and context window.

context window

- *"partner abroad who can accommodate 200M USD for Investment"*

- Context window: Specified by a width around a word.
  - Example: Context window of width 2 around "can".
- Co-Occurrence: Have a co-occurrence between "can" and every word in the context window around "can".

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	…	1	0
taste	1	0	…	2	1
...	…	…	…	…	…
good	1	2	…	0	0
pears	0	1	…	0	0

Co-occurrence matrix

- d_1=*"apples taste good and are healthy"*
- d_2="*the taste of pears is delicious"*
- d_3="*basil adds a good taste to the dish"*

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	...	1	0
taste	1	0	...	2	1
...	...	...	...	...	...
good	1	2	...	0	0
pears	0	1	...	0	0

Co-occurrence matrix

- d_1="*apples taste good and are healthy*"
- d_2="*the taste of pears is delicious*"
- d_3="*basil adds a good taste to the dish*"

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	…	1	0
taste	1	0	…	2	1
...	…	…	…	…	…
good	1	2	…	0	0
pears	0	1	…	0	0

Co-occurrence matrix

- d_1="*apples taste good and are healthy*"
- d_2="*the taste of pears is delicious*"
- d_3="*basil adds a good taste to the dish*"

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	…	1	0
taste	1	0	…	2	1
...	…	…	…	…	…
good	1	2	…	0	0
pears	0	1	…	0	0

Co-occurrence matrix

- Embedding:
  - Basic: A word t is embedded to its row in the co-occurrence matrix.

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	…	1	0
taste	1	0	…	2	1
...	…	…	…	…	…
good	1	2	…	0	0
pears	0	1	…	0	0

Embedding of "apples"

- Embedding:
  - Basic: A word t is embedded to its row in the co-occurrence matrix.

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	…	1	0
taste	1	0	…	2	1
...	…	…	…	…	…
good	1	2	…	0	0
pears	0	1	…	0	0

Embedding of "pears"

- Embedding:
  - Basic: A word t is embedded to its row in the co-occurrence matrix.

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

	apples	taste	...	good	pears
apples	0	1	...	1	0
taste	1	0	...	2	1
...	...	...	...	...	...
good	1	2	...	0	0
pears	0	1	...	0	0

Embedding of "pears"

- Intuition:
  - "apples" and "pears" have more similar rows than e.g. "apples" and "car".
    - Occur more often with "taste" than "car" does.

Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.

- Embedding:
  - Basic: A word t is embedded to its row in the co-occurrence matrix.

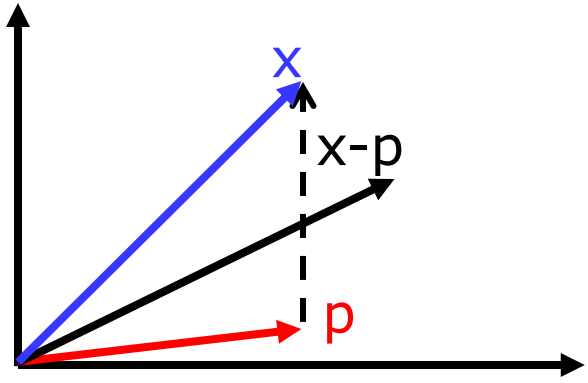
- Issue:
  - Very long vectors!
  - Solution: Dimensionality reduction.
    - One popular approach is based on PCA (Principal Component Analysis).

PCA

- Principal Component Analysis
  - Let $z_1,\ldots,z_k$ be k eigenvectors corresponding to largest eigenvalues of X^TX and project feature vector x to $p_x = \sum_{i=1}^{k}(x^T z_i)z_i$.
- Write x in the new basis:
  - $\hat{x} = \begin{pmatrix} x^T z_1 \\ \vdots \\ x^T z_k \end{pmatrix}$
- On whole data matrix:
  - X′=XZ with $Z = \begin{pmatrix} | & \ldots & | \\ z_1 & \ldots & z_k \\ | & \ldots & | \end{pmatrix}$



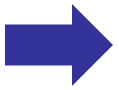
MNIST



Word Embeddings

- Co-Occurrence Matrix for training documents $d_1,\ldots,d_n$:
  - Matrix M with entry per pair of words (t_1,t_2). Counts #co-occurrences (t_1,t_2) in $d_1,\ldots,d_n$.
  - Do eigendecomposition and replace M by MZ to get dxk matrix.
  - Use rows as embeddings of words.

	apples	taste	...	good	pears
apples	0	1	...	1	0
taste	1	0	...	2	1
...	...	...	...	...	...
good	1	2	...	0	0
pears	0	1	...	0	0



apples	0.1	...	-0.1
taste	1.0	...	-1.3
...	...	...	...
good	0.7	...	0.3
pears	0.2	...	0.0

- Main Issues:
  - Slow to compute eigendecomposition on large matrices.
  - Memory heavy to compute eigendecomposition.